

신뢰할 수 있는 데이터 흐름을 설계하는

데이터 엔지니어

예측 가능한 구조를 기반으로, 운영 가능한 데이터 플랫폼을 설계합니다.

프로젝트

분야	역할	프로젝트	기술 스택
데이터	데이터 엔지니어	광고 캠페인 데이터 플랫폼 (개인프로젝트)	Iceberg · Spark · Airflow
	팀장·분석	지게차 충돌 감지·추적 시스템	YOLO · BoT-SORT
	인프라·분석	스마트팜 양액펌프 막힘 예지보전 시스템	AutoEncoder · FastAPI · MQTT
개발	DevOps	이미지 메타데이터 기반 기록 서비스(Log-Land)	EKS · CI/CD
	FE/BE	청년 미래소득 기반 금융 플랫폼	Java · Spring · React
	FE/BE	애완동물 동반 여행 플래너(당동)	SpringMVC · React · Mybatis

한승보

1996년생 · 서울

본질 복잡한 내용을 이해하고 쉽게 정리하는 것을 좋아합니다.

성장 배운 것을 적용하며 더 나은 방법을 찾아갑니다.

01 타임라인

2026 ~ NOW

데이터 · AI

데이터·AI의 서비스화 및 플랫폼

2025

VDI 운영 (실무 1년)

장애 대응 · 부서 간 커뮤니케이션 · 플랫폼 운영 이해

2024

인프라 교육

네트워크 · 클라우드 인프라 학습

2023

개발 교육

언어 · 프레임워크에 대한 이해

02 연락

깃허브 github.com/boseungdl

이메일 96tmdtmd@gmail.com

전화 010-2015-3932

● 프로젝트 01 / 06

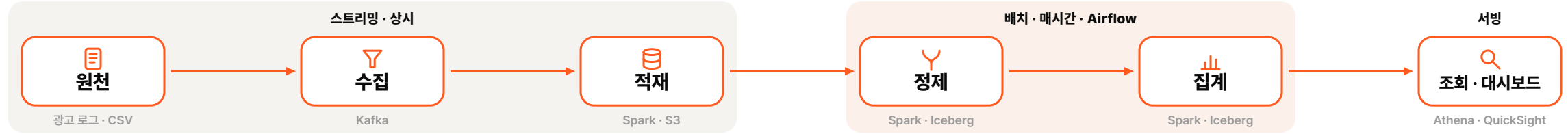
광고 데이터 레이크하우스

데이터

"실시간 광고 이벤트를 최적의 성능과 비용으로 활용할 수 있게 관리하는 데이터 플랫폼"

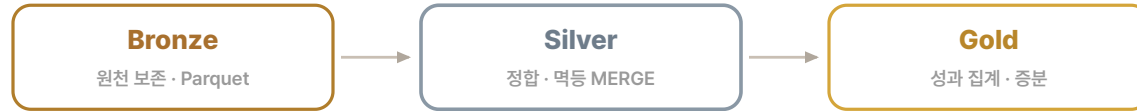
설계 기준: 일 100만 → 1억 이벤트 / 일 1GB → 100GB

전체 아키텍처 · 수집부터 조회까지



메달리온 3계층

보존 · 정제 · 집계, 계층별 최적화



왜 Iceberg?

배치 오케스트레이션 · Airflow DAG

- ① 정제 DAG (매시 :00) : 새 데이터 적재 → 늦게 온 전환 MERGE
- ② 집계 DAG (매시 :30) : 성과 재집계(CTR-CVR-CPA)
- ③ 유지보수 DAG (별도 주기) : 컴팩션 · 스냅샷 만료 · 고아 파일 정리

→ 변경분만 재계산해 매시간 최신 · 읽고 쓰는 양↓ = 비용↓

문제 → 지연 도착

전환은 3일 늦게 도착. 일반 레이크는 반영하려면 파일 통째 재작성 (읽기·쓰기 비용↑)



행을 고칠 수 있는 Iceberg로 한 행에 MERGE

성과 지표 Gold

- CTR 클릭률
- CVR 전환율
- CPA 전환당 비용

→ 광고 효과·예산 판단 근거

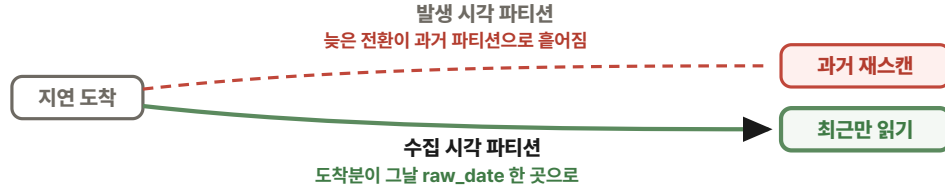
운영 가시성 · 헬스체크

전체 스캔 없이 메타데이터로 즉시 점검

- 신선도 마지막 적재가 언제인지
- 행 수 유입량이 정상 범위인지
- 파일 수 스물파일 폭증은 없는지

→ 이상 징후 조기 감지·장애 예방

01 확장성: 파티션 기준을 발생 시간에서 수집 시간으로



파티션 기준을 발생 시각에서 수집 시각으로 바꿔, 늦게 온 데이터도 그날 파티션 한 곳에만 쌓이게 했습니다. 처음 발생 시각으로 잡았을 땐, 전환이 최대 30일까지 늦게 오면서 오늘 도착분이 과거 날짜 파티션 수십 개로 흩어졌습니다. 최근 파티션만 봐선 새 데이터를 못 챙기고 과거까지 다시 훑어야 했는데, 수집 시각 기준에선 도착분이 발생일과 무관하게 그날 raw_date에 모여, 과거 파티션은 다시 건드릴 일이 없고 정제 때도 최근 파티션만 읽어 스캔을 줄일 수 있는 구조가 됐습니다. 발생 시각은 event_timestamp로 남겨 Silver 분석에서 사용했습니다.

빠르게 받아내는 게 raw의 일이지만, 쌓는 방식 하나가 정제의 부담까지 바꿨습니다.

02 비용 최적화: 스몰 파일 병합으로 조회 비용 절감



조회 비용을 줄이려고 스몰 파일을 주기적으로 병합했습니다. Silver는 매시간 증분 MERGE로 갱신돼 늦은 전환이 여러 파티션에 소량씩 쌓이고 MOR 특성상 delete 파일까지 늘어, 그대로 두면 조회 때 LIST-GET-메타 파싱이 파일 수만큼 불어납니다. 그래서 rewrite_data_files로 128MB 단위로 병합하고, 스냅샷 만료·고아 파일 정리도 같은 유지보수 배치로 함께 돌렸습니다. 스캔을 건너뛰는 프루닝은 Iceberg가 파티션·파일 통계를 해주는데, 파일이 잘게 흩어지면 파일별 min/max 범위가 겹쳐 잘 걸리지 않습니다. 병합으로 파일을 정돈해 그 프루닝이 제대로 먹히게 하는 게 제 몫이었습니다.

저장 구조를 정할 땐 쓰는 속도만이 아니라, 읽는 비용도 같이 봐야 했습니다.

03 정확성: 중복 유입에도 결과를 하나로



재처리해도 전환 수가 흔들리지 않도록, 중복을 단계별로 흡수하는 역등 구조로 설계했습니다. 전환은 CVR·CPA로 이어지는 광고비 판단의 핵심이라, 한 건이라도 부풀면 예산 결정이 틀어집니다. 그런데 Kafka가 at-least-once라 재기동·재시도 때 같은 eventId가 다시 들어와, 그대로 집계하면 전환이 실제보다 많게 잡힙니다. checkpoint 오프셋 커밋은 유실만 막을 뿐 이 중복은 못 막습니다. Bronze는 원본을 손대지 않고 그대로 받는 계층이라 중복을 여기서 거르지 않고, 정확성은 Silver에서 흡수하기로 했습니다. raw에 같은 event_id가 여러 벌 쌓이면 그대로 MERGE할 때 한 타깃 행에 여러 소스가 부딪혀 실패하므로, 조립 땐 ingest_ts로 event_id별 최신 한 건만 남겨 소스를 유일하게 만든 뒤 event_id로 MERGE했습니다. 없으면 새로 넣고 있으면 그 행을 갱신하니, 몇 번을 다시 돌려도 결과가 같습니다 (역등). MERGE는 신규 INSERT와 지연 UPDATE를 태스크로 나눠, 어느 쪽이 어긋나는지 Airflow에서 바로 보고 실패한 쪽만 재시도하게 했습니다.

중복은 원천 봉쇄하기보다, 몇 번 들어와도 같은 결과가 되게 흡수하는 게 현실적이었습니다.

성과 · 결과 OUTCOMES

확장성	지연 데이터도 재적재 없이 흡수 · 규모 확장 대비 수집 시각 파티션 · 증분 읽기 기반 {확정 시} 파티션당 파일 수 일정
비용 최적화	조회 스캔량 · S3 호출 비용 ↓ 128MB 병합 + 파티션·파일 프루닝 {확정 시} compaction 전후 파일 수 ↓
정확성	재처리해도 집계 결과 동일 (역등) checkpoint · dedup · event_id MERGE {확정 시} 재처리 전후 전환 수 동일

● 다음 프로젝트 02 / 06

지게차 충돌 위험 감지 시스템

데이터

"지게차-보행자 충돌 위험을 CCTV로 실시간 감지해 사고 전에 경고하는 안전 시스템"

설계 환경: Unity 합성 영상 · RTSP 30fps 매 프레임 추론 · 충돌 2~3초 전 경고

문제 정의

"지게차-보행자 충돌 위험이 사전에 감지되지 않아, 사고가 난 뒤에야 인지됩니다"

1

배경 · 분석 필요성

- **배경** 지게차 재해자는 2021~22년 **2,559명**(2022년 1,163명). 그중 보행자와의 '**부딪힘**'이 **56%**로 1위. 원인은 운전자의 시야 확보 부족.
- **배경** 제조업(50인 미만) 사망사고 **기인물 1위가 지게차**. 사람이 다치고 나서야 인지되는, 막을 수 있었던 사고.
- **분석 필요성** 사고 후 기록이 아니라 **사람이 알아채기 전에 미리 경고**가 목표. 단순 TTC는 멈춘 물체가 ∞로 잡히고 접근 각도를 못 봐, **거리·방향까지 결합한 위험 분석**이 필요.

2 AI로 사고 전 예측

자체 학습 모델
forklift_v1.pt · Unity 합성

검출

YOLO11

역할 영상에서 지게차-사람을 실시간으로 찾아냄

추적

BoT-SORT

역할 같은 객체에 ID를 부여해 프레임 간 추적

예측

등속 예측 (CV)

역할 속도로 1~2초 뒤 객체들의 위치를 직선 예측

위험 평가

TTC

역할 충돌까지 남은 시간(TTC)-거리 기반으로 위험도 산출

운영자 알람
실시간 · DB 적재

설계 의도 무거운 단일 모델 대신 **단계마다 가볍고 검증된 기법**을 골라 **30fps 실시간**을 지키면서, 정지·가림처럼 **놓치기 쉬운 상황**까지 잡습니다.

내 역할

팀장 · 검출·추적

담당

팀을 이끌며 검출(YOLO11)·추적(BoT-SORT)을 담당했습니다. 오탐 제거부터 끊임 없는 추적, 실시간 추론 안정화까지

핵심 과제 · 해결

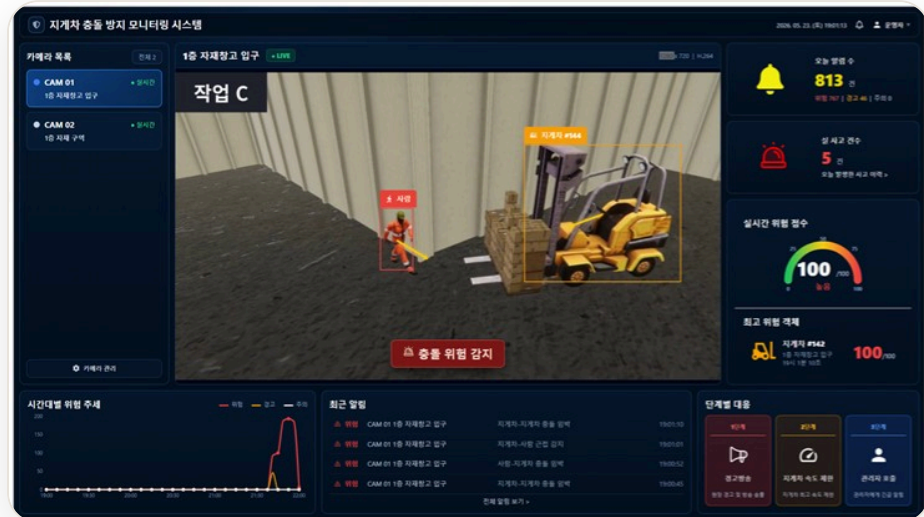
검출: 오탐을 지우면 진짜를 놓치는 문턱 딜레마

오탐을 지우면 미탐이 늘던 검출 딜레마를, 문턱을 역할별로 나눠 해결했습니다. 처음엔 검출 확신도(conf)를 0.5까지 올려 트럭·상자를 지게차로 잡는 오탐을 지우고 중복 박스도 NMS iou 0.7→0.45로 합쳤는데, 이번엔 회전하거나 가려진 순간 확신도가 떨어진 진짜 객체까지 걸러져 놓쳤습니다. 그래서 입구 conf는 0.3까지 다시 낮춰 약한 검출도 일단 트래커로 넘겼습니다. 이미 추적 중인 객체는 track_low_thresh 0.01로 아주 흐린 검출로도 같은 ID를 유지해 가림 순간에도 놓치지 않게 하고, 새 객체 등록만 new_track_thresh 0.65로 엄격하게 걸렀습니다. 0.65를 넘겨 후보가 돼도 바로 다음 프레임에 다시 잡혀 매칭돼야 정식 트랙이 됩니다(DeepSORT가 n_init=3으로 두는 연속 확인 관문이고, BoT-SORT는 2프레임 고정입니다). 진짜 객체는 화면에 수십 프레임 계속 있으니 이 관문을 자연히 통과하고, 한두 프레임 반짝하는 오탐은 여기서 걸러집니다. 놓치지 않으면서 오탐도 뜨지 않는 검출로 안정화했습니다.

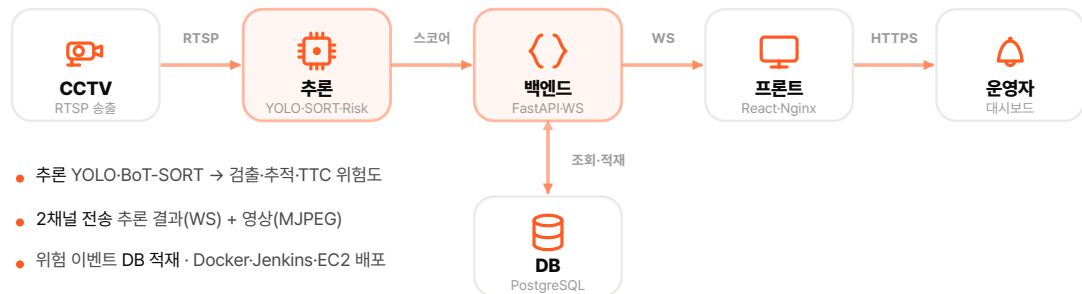
추적: 회전·가림마다 ID가 바뀌는 ID 스위칭

회전하거나 가려질 때마다 같은 지게차에 새 ID가 붙던 ID 스위칭을, 매칭 성공률을 높여 잡았습니다. 원인은 매칭 실패였습니다. 트래커는 "예측한 위치와 새 검출이 겹치는가"로 같은 객체를 이어가는데, 가려지면 검출이 사라지고 회전하면 박스 모양과 확신도가 무너져, 짝을 못 찾은 검출이 새 ID를 받았습니다. 트래커부터 바꿨습니다. 흐려진 저신뢰 검출을 버리는 DeepSORT에서, 그걸 2차 매칭으로 재활용해 트랙을 잇는 ByteTrack으로 옮겨 회전·가림의 끊김을 줄였고, 외형 매칭(ReID)까지 없을 수 있는 BoT-SORT로 정착했습니다. 그 위에서 match_thresh를 0.6→0.9로 올려 덜 겹쳐도 같은 객체로 인정하게 했고, 잠깐 사라진 트랙은 Lost로 보관하며 칼만 예측으로 위치를 이어가다 재등장하면 같은 ID로 복구시켰습니다. 보관을 길게 두면 유령 박스가 남아 track_buffer는 10프레임으로 줄였고, 재등장 위치가 예측과 어긋난 객체는 ReID로 생김새를 대조해 재확인했습니다. 그 결과 회전·가림을 지나도 ID가 유지돼 속도와 위험도 계산이 끊기지 않았습니다.

결과물 · 실시간 모니터링 대시보드 현장 안전관리자용



시스템 아키텍처 메인 런타임



● 다음 프로젝트 03 / 06

스마트팜 양액펌프 막힘 예지보전

데이터

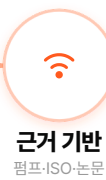
"양액펌프 막힘의 전조를 센서 데이터로 미리 잡아 가동 중단 전에 알리는 예지보전 시스템"

데이터 인프라·분석·비지도 AutoEncoder 이상탐지 · 45개 변수를 4개 도메인으로 분리

문제 정의

"양액펌프 막힘의 미세한 전조를 사전에 잡지 못해, 작물 손실과
과잉 정비 비용으로 이어집니다"

AI로 막힘 전 예측



도메인별 학습

모터 AE

유압 AE

양액 AE

구역 AE

위험도 통합

종합 위험 판정
어느 도메인·왜

운영자 알람
실시간·대시보드

배경·분석 필요성

- 배경 양액펌프는 작물에 영양분을 공급하는 스마트팜의 '심장'. **24시간 안정 가동**이 수확량·품질에 직결되는 핵심 설비.
- 배경 막힘은 노즐에 염분·이물질이 **서서히 쌓이며 진행**돼, 사후 대응·주기적 과잉 세척에 의존 → **생육 타격·불필요 정비 비용**.
- 분석 필요성 EC·pH(화학) 센서는 오염·드리프트·반응 지연으로 조기 탐지에 부적합 → **압력·유량·전류 등 물리 지표**로 전조 포착. 공개 고장 데이터가 없어 **학습셋을 직접 구축**해야 함.

왜 비지도 AutoEncoder?

- 고장 사례 데이터가 드물어 지도학습이 어려움 → **정상 패턴만 학습**하고 **재구성 오차**로 이상을 탐지.

왜 4개 도메인인가?

- 45개 변수를 한 모델로 합치면 노이즈가 간섭 → **모터·유압·양액·구역**으로 분리해 정확도 ↑·**원인 짚기**.

왜 데이터를 직접 만들었나?

- 공개 펌프 데이터가 없어 **공공데이터+물리식**으로 합성(45종·90일). EC·pH 대신 **물리지표 + system_resistance**.

내 역할 인프라 · 분석

담당

MQTT 수집 → 추론 → 실시간 서비스 인프라와 데이터·모델을 담당했습니다. 학습셋 직접 구축부터 4-도메인 AutoEncoder 설계·검증까지

핵심 과제 · 해결

데이터: 학습에 쓸 고장 데이터가 없음

정답 라벨이 없어 정상만 학습하는 비지도 AutoEncoder로 방향을 잡고, 학습 데이터 자체가 없어 '근거 있는 합성'으로 직접 만들었습니다. 공개 고장 데이터가 없어 지도학습이 불가능했고 EC·pH 센서는 막힘을 수 시간~수 일 늦게 드러냈기에, 정상 패턴을 학습해 유량·압력·전류의 관계로 전조를 잡았습니다. 합성은 임의값이면 근거가 없으므로 환경→제어→센서→고장 순으로 쌓고 막힘(clog) 하나가 여러 센서를 함께 바꾸도록 설계했으며, 정상값은 실제 3마력 양액공급기·계수는 ISO·논문·딸기 환경은 AI-Hub 실측치를 앵커로 삼아 임의 수치를 최소화했습니다.

모델: 변수 45개를 한 모델에 넣어 생긴 간섭

모터·수력·양액·구역 4개 독립 AutoEncoder로 나눠 간섭을 없애고, 원인은 RCA로, 라벨 없는 검증은 동적 임계값으로 풀었습니다. 45변수를 한 모델에 넣으면 화학(EC·pH) 노이즈가 물리 판정에 간섭하므로, 도메인을 나누고 피쳐는 상관 중복 제거 + 도메인 타깃 SHAP으로 고르되 핵심 센서는 반드시 포함했고, 시간·상태 피쳐는 채점에서 빼 오탐을 줄였습니다. 계절·시간 드리프트엔 도메인별 동적 임계값(기동 구간 제외)으로 대응했고, 독립 seed 고장으로 정직하게 재니 검출은 단일센서 기준선과 동등하되 오탐률을 3.1%→1.8%로 약 1.7배 낮췄습니다.

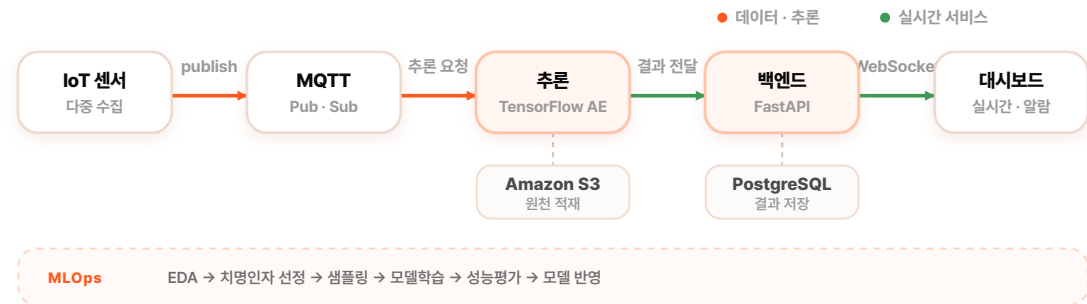
안정화: 이상 점수가 한 지표에 쏠린 문제

이상 점수가 유량감소율 한 지표에 쏠린 것을 단서로, 극단값이 정규화를 어긋나게 한 원인을 찾아 게이트로 해결했습니다. 펌프 기동 순간 기준유량이 0에 가까워지면서 유량감소율이 비정상적으로 큰 값이 되었고, 이 소수의 극단값이 MinMax 정규화 범위를 틀어지게 해 AutoEncoder의 재구성 오차를 크게 키우고 있었습니다. 운전 상태·기준값·범위의 3단계 게이트로 이 극단값을 걸러, 지표 쏠림을 없애고 학습을 안정화했습니다.

결과물 실시간 모니터링 대시보드



시스템 아키텍처 센서 → 추론 → 실시간 서비스



● 다음 프로젝트 04 / 06

Log-Land · 이미지 메타데이터 기반 기록 플랫폼

개발

서비스 개요

사진 EXIF 메타데이터(시간·위치)를 읽어 지도·타임라인에 추억을 자동 기록하는 클라우드 다이어리입니다.

내 역할

업로드·저장 설계 · CI/CD. 이미지 업로드 파이프라인을 설계하고, 배포 과정을 자동화했습니다.



트러블슈팅

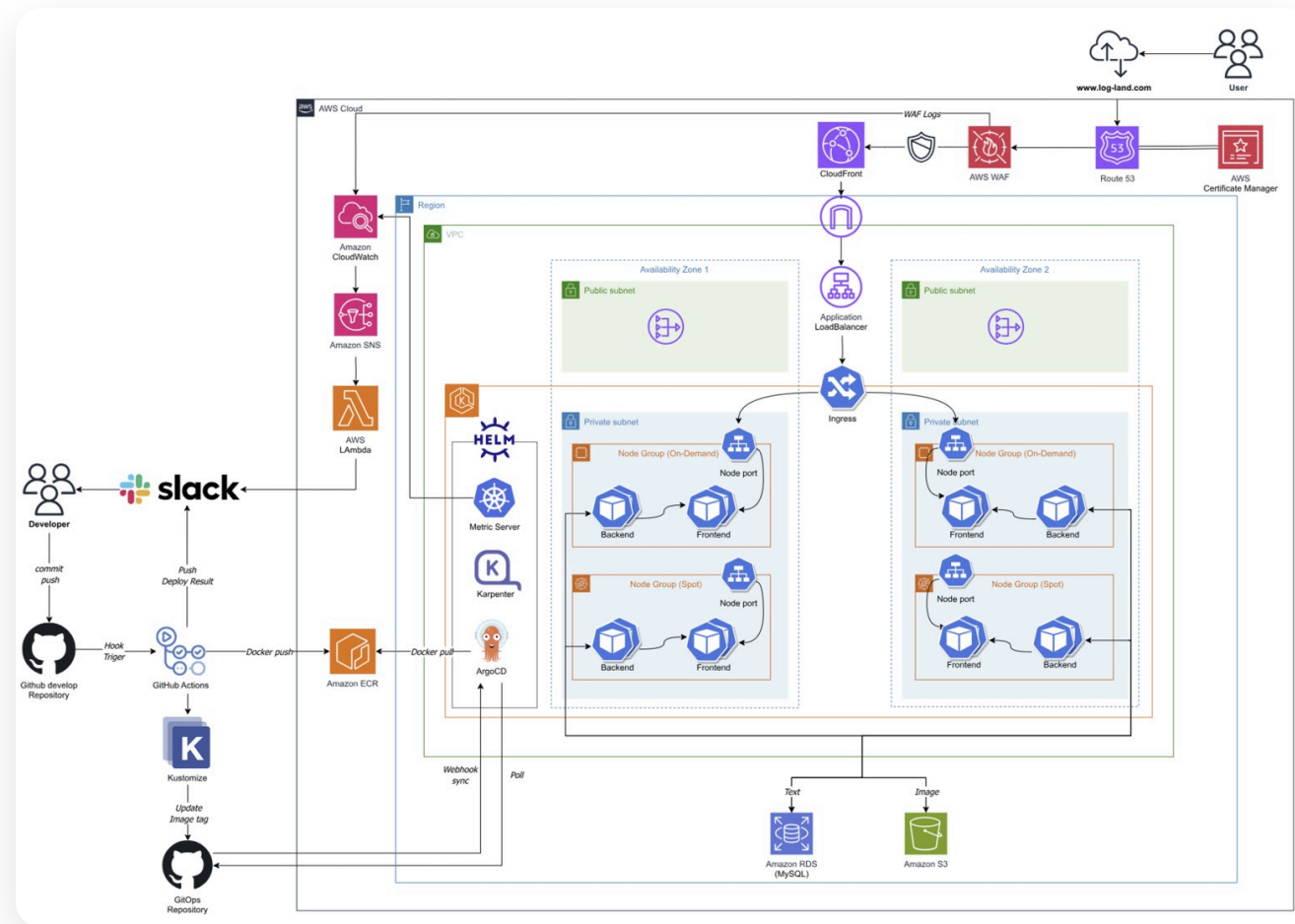
■ 설정 외부화: 환경이 달라도 같은 이미지로 배포

로컬에서 잘 돌던 화면이 EKS에 올리자 프론트에서 백엔드 API를 부르지 못했습니다. 처음엔 Ingress 라우팅만 손보면 될 문제로 봤습니다. 그러나 실제 원인은 코드가 로컬 환경을 전제하고 있다는 데 있었습니다. 프론트·백엔드 주소가 코드에 박혀 있었고 CORS도 모든 출처를 열어둔 상태라, 이미지는 하나인데 띄우는 곳에 따라 동작이 달라졌습니다. 그래서 주소를 전부 환경변수로 빼 배포 시점에 주입하고, 허용 출처는 와일드카드 대신 그 환경의 프론트 도메인 하나로 좁혔습니다. Service-Ingress 라우팅도 그 주소 체계에 맞춰 정리했습니다. 그 결과 같은 이미지를 어디에 올려도 주입값만 바꾸면 그대로 댔습니다. 컨테이너 이미지는 자기가 어디서 도는지 몰라야 하고, 환경은 밖에서 주입돼야 한다는 걸 배웠습니다.

■ GitOps: PR 머지 한 번으로 EKS까지 자동 배포

처음엔 GitHub Actions에서 kubectl로 클러스터에 직접 배포(push)하면 충분하다고 봤습니다. 그러나 자격증명을 CI에 보관해야 하는 보안 부담과 배포 이력·롤백이 코드와 따로 노는 점을 가볍게 본 게 잘못된 판단이었습니다. 이후 ArgoCD GitOps(pull)로 전환해, Actions는 Build-ECR Push-image tag commit까지만 맡고 동기화는 ArgoCD가 Git 상태를 보고 수행하도록 분리했습니다. 그 결과 배포 상태가 항상 Git과 일치(선언적)하고 롤백은 revert 한 번, 자격증명은 클러스터 안에만 남아 보안과 재현성을 함께 확보했습니다. 자동화는 '돌아가게'가 아니라 '안전하게 추적·복구되게' 만드는 것이라는 걸 배운 작업이었습니다.

인프라 아키텍처



● 다음 프로젝트 05 / 06

Peoch(피치) · 청년 미래소득 기반 금융 플랫폼

개발

서비스 개요

신용정보가 아닌 청년의 미래 소득 가능성을 평가해 투자하는 금융 플랫폼입니다. 고정 원리금이 아닌 예상 수익 비례 상환으로 청년 자립을 돕고, 청년↔기업 ESG 선순환을 설계했습니다.

내 역할

커스텀 카드·혜택 도메인 백엔드를 맡아 카드 발급, 혜택 구독·해지, 결제 이력 조회 API를 설계·구현했습니다. 소득 추정 파이프라인(스펙 정규화 → LLM → 헨가 할인)은 팀과 함께 설계했습니다.

BE Spring Boot · JPA · MariaDB · REST API

팀 Security/JWT · Redis · Batch · iText · LLM 파이프라인

트러블 슈팅

● 혜택 해지에 함께 지워진 원본 혜택

카드와 혜택을 잇는 매핑에 삭제가 전파되도록 걸어 둔 탓에, 구독 혜택 하나를 끊자 모두가 쓰는 혜택 원본까지 같이 지워졌습니다. → 매핑은 매핑만 지우도록 삭제 전파를 제거하고 카드 참조를 지연 로딩으로 바꿔, 해지가 연결된 한 행만 건드리게 했습니다.

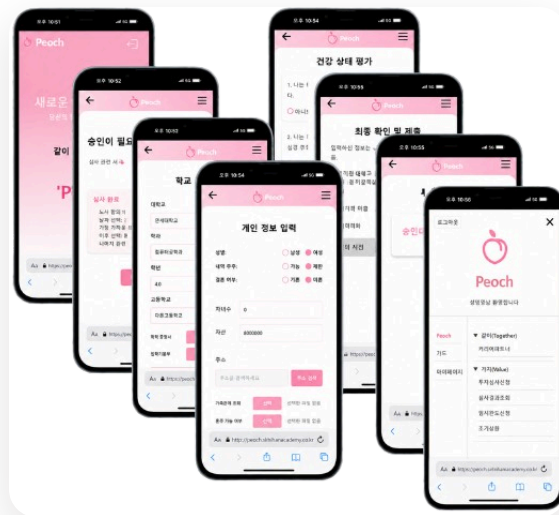
● 혜택 목록 조회마다 늘어나던 쿼리

카드에 적용된 혜택을 엔티티 그대로 응답하다 보니 혜택 하나마다 추가 쿼리가 나갔고, 지연 로딩 상태의 객체가 그대로 직렬화되며 응답 형태도 흔들렸습니다. → fetch join으로 혜택을 한 번에 가져오고 응답을 DTO로 좁혀, 화면이 쓰는 필드만 내려가게 했습니다.

● 주소로 열리지 않던 업로드 이미지

카드 정보·디자인·이미지를 한 요청으로 받아 이미지를 리소스 폴더에 저장했는데, 실행 중인 애플리케이션은 빌드 시점에 묶인 리소스만 바라봐서 새로 올라온 파일이 주소로 열리지 않았습니다. → 정적 리소스 요청을 저장 디렉터리와 같은 기존의 파일시스템 경로에 매핑해 쓰기 경로와 읽기 경로가 한 지점을 가리키게 했고, DB에는 절대 URL 대신 매핑 기준의 상대 경로만 남겨 저장 위치가 바뀌어도 데이터는 두고 설정만 바꾸면 되게 했습니다.

결과물 · 화면



주요 기능

인증·인가

JWT 발급·검증·권한 + 로그아웃 토큰 블랙리스트(Redis)

소득추정

스펙 정규화(Perplexity·Batch) → 프롬프트 LLM으로 연차별 예상소득 생성 → 헨가 할인으로 투자금 산출

투자·심사

서류 제출 · 계약서/명세서 PDF(iText) · 심사 결과 · 지원금 변경 · 엑시트

카드·결제

카드 신청·발급 · 카멜레온 디자인 · 혜택 구독/해지 · 결제 이력 조회 · 월 스케줄러 한도 차감 · POS 결제

● 다음 프로젝트 06 / 06

댕동 · 반려견 동반 여행 플래너

개발

서비스 개요

반려견과 여행을 떠나는 사용자를 위한 맞춤형 여행 플래너로, 일정을 계획하고 여행 기록·포토카드로 추억을 남깁니다. 반려동물 산업 성장이 배경, 기존 숙소 예약 위주 서비스와 달리 여행 일정 계획 + 동행자 관리로 차별화했습니다.

내 역할

6인 팀에서 게시판/커뮤니티를 담당해 게시글·댓글 CRUD(MyBatis postMapper)와 좋아요, 이미지 업로드를 직접 구현했습니다. 데일리 스크럼·KPT 회고·WBS로 팀 진행을 관리했습니다.

FE HTML · CSS · JS · JSP

BE Spring MVC · MyBatis · MySQL

가능 WebSocket · AWS S3 · Kakao/Google OAuth

트러블 슈팅

● 사진 장수만큼 부풀던 좋아요 수

게시글·좋아요·이미지를 한 번에 조인하니 이미지가 여러 장인 글은 행이 곱해져, 좋아요 수가 사진 장수만큼 부풀어 올랐습니다. → 좋아요는 미리 집계한 서브쿼리로 만들어 글에 한 행씩 붙이고, 목록에는 대표 이미지 한 장만 뽑도록 조인을 나눠 집계와 표시를 분리했습니다.

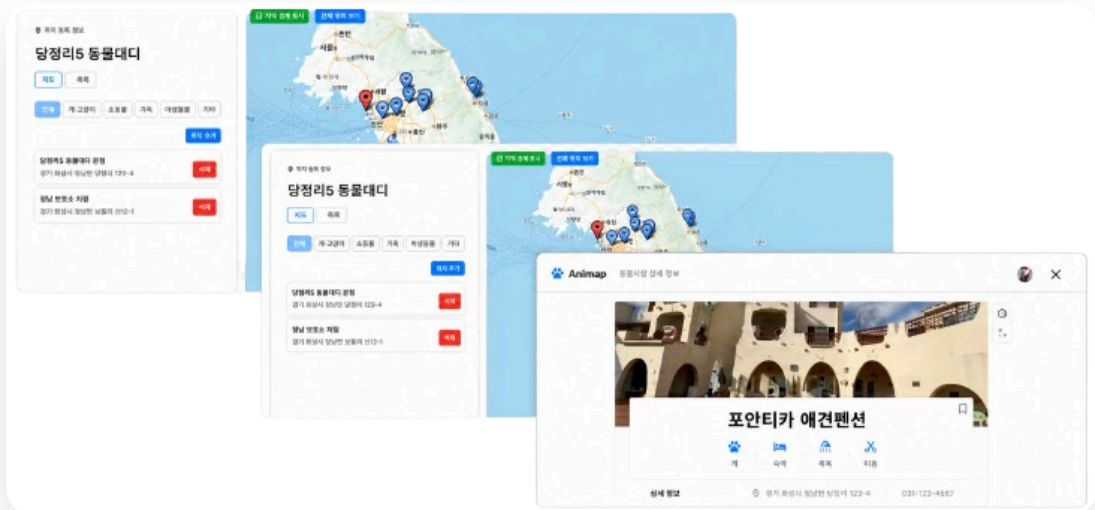
● 중복으로 들어갈 수 있던 좋아요

좋아요는 있으면 지우고 없으면 넣는 방식이라, 요청이 겹쳐 들어오면 둘 다 “아직 없다”를 보고 각각 저장할 수 있었습니다. → 좋아요 테이블의 키를 회원·게시글 복합 키로 잡아 같은 조합이 두 행으로 남는 것을 DB에서 차단하고, 애플리케이션 판단이 어긋나도 좋아요 집계는 흔들리지 않게 했습니다.

● 컨트롤러까지 오지 않던 업로드 파일

게시글을 쓰면서 이미지를 함께 보내는데, 컨트롤러에서는 파일이 잡히지 않아 업로드 자체가 성립하지 않았습니다. → 파일 업로드를 처리하는 multipartResolver의 등록 위치를 애플리케이션 공통 설정으로 옮겨, 업로드 요청이 컨트롤러까지 파일로 전달되게 했습니다.

결과물 · 화면



주요 기능

여행플랜

여행 일정을 입력하면 자동으로 계획 구성 · 일정 추가 시 동적 버튼 생성으로 직관적 표시

동행자

팔로잉/팔로워로 동행자를 추가·삭제하고 일정을 함께 공유

장소검색

카카오맵 API로 반려견 동반 가능 장소를 검색·추천·즐거찾기

커뮤니티

게시판 게시글·댓글과 이미지 업로드로 사용자 간 소통

실시간채팅

WebSocket 기반 동행자 간 실시간 채팅(룸·세션 관리)